Persistent

# Artifacts and Caching in GitLab CI/CD

Optimize Your Pipelines with Reusability & Speed

# Agenda

1. **What Are Artifacts and Caches?**

   Understanding the fundamentals of temporary storage in CI/CD

2. **Differences Between Artifacts vs Cache**

   Key distinctions in purpose, lifecycle, and accessibility

3. **Artifacts: Storing Build Outputs**

   Implementation and use cases for build artifacts

4. **Cache: Speeding Up Pipelines**

   Techniques for reducing build times with intelligent caching

5. **Hands-On: Using Cache & Artifacts**

   Practical examples for implementing both strategies

6. **Best Practices**

   Optimization tips and recommended approaches

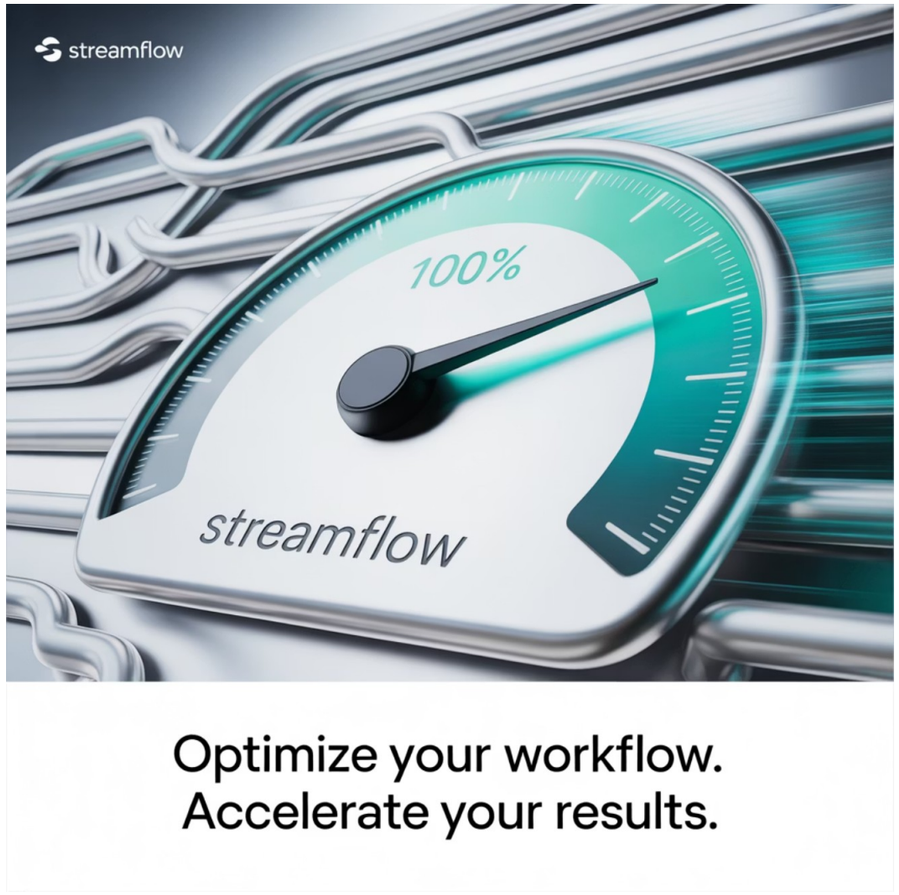
Persistent

# What Are Artifacts and Cache?


**Artifacts**

Transfer files between jobs/stages within a single pipeline run

Allow you to pass compiled binaries, reports, or other build outputs forward in your pipeline




**Cache**

Reuse dependencies or tools across pipeline runs to speed up builds

Store files like node_modules, vendor packages, or compiled assets to avoid expensive rebuilds

Persistent

# Artifacts – Build Output Sharing

Artifacts allow you to pass build outputs between stages in your pipeline. They're automatically downloaded in subsequent jobs and can be accessed through the GitLab UI.

**Common Use Cases:**

- Compiled binaries
- Test reports
- Generated documentation
- Deployment packages

```
build:  stage: build  script:    - make build  artifacts:    paths:      - build/    expire_in: 1 week    test:  stage: test  script:    - test -d build/  # Verify artifacts exist    - ./run_tests.sh
```

Artifacts are retained for 30 days by default but can be customized with **expire_in** parameter.

Persistent

# Cache – Accelerating Pipelines


```
cache:  key: ${CI_COMMIT_REF_SLUG}  paths:    - node_modules/    - .yarn    - vendor/  policy: pull-push
```

Cache keys can be defined per-branch, per-job, or based on content hashes to ensure cache validity.

Policies control when cache is updated: **pull-push** (default), **pull**, or **push**.

**Cache Benefits**

- Reduce build times dramatically

Persistent

# Artifacts vs Cache – Key Differences

### Retention & Lifecycle

**Artifacts:** Retained for 30 days (configurable) after pipeline completion

**Cache:** Retained indefinitely per runner until manually cleared or keys change

### Accessibility

**Artifacts:** Downloadable from GitLab UI, accessible across stages

**Cache:** Not downloadable, only accessible by pipeline jobs

### Purpose

**Artifacts:** Store build outputs, reports, deployable packa

**Cache:** Speed up builds by preserving dependencies, libraries, compiled assets

### Scope

**Artifacts:** Available within a single pipeline run

**Cache:** Available across multiple pipeline runs on the same runner

Persistent

# Hands-On: Caching NPM Modules

```
stages:  - install  - buildinstall-deps:  stage: install  script:    - npm install
cache:    key: $CI_COMMIT_REF_SLUG-node    paths:      - node_modules/    policy: pull-
pushbuild-app:  stage: build  script:    - npm run build  cache:    key:
$CI_COMMIT_REF_SLUG-node    paths:      - node_modules/    policy: pull
```

**Key Implementation Points:**

- Using branch-specific cache keys prevents conflicts

First job uses **pull-push** to update cache

Subsequent jobs use **pull** to save upload time

- Cache is specific to each runner

This approach can reduce build times from minutes to seconds in dependency-heavy projects.


Persistent

# Hands-On: Passing Build Artifacts

```
build:  stage: build  script:    - npm run build  artifacts:
 paths:      - dist/    expire_in: 1 weektest:  stage: test
script:    - npm test    - ls dist/  # Verify artifacts
deploy:  stage: deploy  script:    - cp -r dist/*
/var/www/html/  environment: production
```

**Implementation Flow:**

**Build Stage**

Creates build artifacts in dist/ directory

**Test Stage**

Accesses artifacts automatically

**Deploy Stage**

Deploys verified artifacts

Persistent

# Best Practices

1

## Use Appropriate Tool for the Job

- Use artifacts for sharing files between stages
- Use cache for dependency folders like node_modules/
- Never use cache for sensitive data

2

## Optimize Cache Effectiveness

- Use specific cache keys based on lockfiles (e.g., package-lock.json)
- Clear cache manually when dependencies change significantly
- Consider multiple, smaller caches instead of one large cache

3

## Manage Storage Efficiently

- Set appropriate expiration for artifacts
- Avoid caching large build outputs—use artifacts instead
- Use .gitignore patterns to exclude unwanted files

4

## Monitor Performance

- Track pipeline execution times before and after caching
- Watch for diminishing returns with very large caches
- Consider using GitLab CI/CD metrics to optimize

# Q&A / Lab Support

## Need Help With Your CI Configuration?

Let's debug your .gitlab-ci.yml together!

- Optimizing cache configuration
- Setting up artifact dependencies
- Troubleshooting pipeline performance issues
- Implementing branch-specific caching strategies

Documentation: gitlab.com/docs/ci/caching


Persistent

Persistent

# Thank you